

Wybór systemu: **Sklep z Kartami Pokemon**

1. System umożliwia dodawanie kart do oferty, wyświetlanie dostępnych kart oraz ich zakup przez klientów.
2. Opis klas:
 - Klasa *cards* - reprezentuje pojedynczą kartę w sklepie :
 - Pola prywatne: *name,type,price,isavailable*
 - Metody: *buy* – zakup karty, *show* – wyświetlenie informacji , *isAvailable*-sprawdzenie dostepnosc
 - Konstruktor – inicjalizuje nazwe, typ,cene, dostepnosc domyslnie ma wartosc true
 - Klasa *client* – reprezentuje klienta sklepu
 - Pola prywatne: *name*
 - Metody: *show* – wyświetlenie danych klienta
 - Konstruktor – inicjalizuje imię klienta
 - Klasa *shop* – zarządza klientami I kartami – jest to główny element systemu
 - Pola prywatne – *card* jest to wektor obiektów *cards*, *clients* jest to wektor obiektów *client*
 - Metody: *addCard*, *addClient*,*buycard*

3.Relacja kompozycji:

Klasa **shop** zawiera w sobie kolekcje obiektów klas **cards** oraz **client**(pola **card**) I **clients**(typu **vector**).

Oznacza to ,że sklep jest właścicielem tych obiektów I zarządza ich cyklem życia jest to relacja kompozycjo. Usunięcie obiektu **shop** powoduje usunięcie wszystkich przechowywanych kart I klientów

4.Enkapsulacje:

Wszystkie pola we wszystkich klasach sa prywatne. Dostęp do nich jest możliwy tylko I wyłącznie poprzez metody publiczne:

- Stan karty (**isavailable**) zmienia się tylko przez metodę **buy()**, a nie bezpośrednio z zewnątrz
- Lista kart w sklepie (**card**) jest chroniona – można ją modyfikować tylko przez **addCard()** i **buycard()**

5.Kolekcja obiektów:

W klasie **shop** zastosowano wektor **cards** oraz wektor **client** jako kolekcje obiektów. Pozwala to na dynamiczne dodawanie dowolnej liczby kart i klientów w trakcie działania programu.

6.Podejście proceduralne vs obiektowe:

	Proceduralne	Obiektowe
Dane	Oddzielne tablice(nazwy,ceny,dostępność)	Zgrupowane w klasie cards
Funkcje	Globalne, operujące na tablicach	Metody powiązane z obiektem
Bezpieczeństwo	Brak ochrony danych	Enkapsulacja chroni pola
Rozbudowa	Trudna[zmiany w wielu miejscach]	Łatwiejsza[modyfikacja klasy]

W podejściu proceduralnym należałoby utrzymywać oddzielne tablice dla każdej właściwości karty I przekazywać je do funkcji. Podejście obiektowe grupuje dane I operacje razem co zwiększa czytelność I bezpieczeństwo kodu